Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №3

з дисципліни
«Об’єктно-орієнтоване програмування»
на тему
«Розробка інтерфейсу користувача»

Виконала:

Студентка групи ІМ-55

Долгіх Дар’я Михайлівна
Номер у списку групи: 7
Перевірив:
Рекечинський Дмитро Олександрович

Київ 2026

Лабораторна робота №3. Розробка інтерфейсу користувача

Мета: Мета роботи – отримати вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів, запрограмувавши графічний інтерфейс користувача.

Завдання:

1. Створити у середовищі розробки проект з ім’ям Lab3.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт

Масив об'єктів: статичний масив, оскільки 8 mod 3 = 2. Розмір N = 8 + 100 = 108 об'єктів.

"Гумовий" слід: суцільна лінія чорного кольору, оскільки 8 mod 4 = 0.

Увід прямокутника: по двом протилежним кутам, оскільки 8 mod 2 = 0.

Відображення прямокутника: чорний контур прямокутника без заповнення, оскільки 8 mod 5 = 3.

Увід еліпсу: від центру до одного з кутів охоплюючого прямокутника, оскільки 8 mod 2 = 0.

Відображення еліпсу: чорний контур із кольоровим заповненням (світло-зеленим), оскільки 8 mod 5 = 3 та 8 mod 6 = 2.

Позначка поточного типу об'єкту: в меню "Об'єкти", оскільки 8 mod 2 = 0.

Коди

Lab3.py

```
import tkinter as tk
from shapes import PointShape, LineShape, RectShape, EllipseShape

class ToolTip:
    def __init__(self, widget, text):
        self.widget = widget
        self.text = text
        self.tip_window = None
        self.widget.bind("<Enter>", self.show_tip)
        self.widget.bind("<Leave>", self.hide_tip)
```

```
    def show_tip(self, event=None):
        if self.tip_window or not self.text:
            return
        x = self.widget.winfo_rootx() + 20
        y = self.widget.winfo_rooty() + 25
        self.tip_window = tw = tk.Toplevel(self.widget)
        tw.wm_overrideredirect(True)
        tw.wm_geometry(f"+{x}+{y}")
        label = tk.Label(
            tw, text=self.text, background="#ffffe0",
            relief=tk.SOLID, borderwidth=1, font=("Tahoma", "8", "normal")
        )
        label.pack(ipadx=2, ipady=1)

    def hide_tip(self, event=None):
        tw = self.tip_window
        self.tip_window = None
        if tw:
            tw.destroy()

class EditorApp:
    def __init__(self, root):
        self.root = root
        self.current_tool = "Прямокутник"

        self.shapes = []
        self.max_shapes = 108

        self.start_x = None
        self.start_y = None
        self.rubber_id = None
        self.tool_buttons = {}

        self.setup_menu()
        self.setup_toolbar()

        self.canvas = tk.Canvas(root, bg="white")
        self.canvas.pack(fill=tk.BOTH, expand=True)

        self.canvas.bind("<ButtonPress-1>", self.on_press)
        self.canvas.bind("<B1-Motion>", self.on_drag)
        self.canvas.bind("<ButtonRelease-1>", self.on_release)

        self.update_menu_and_title()
```

```python
    def set_tool(self, tool_name):
        self.current_tool = tool_name
        self.update_menu_and_title()

    def update_menu_and_title(self):
        self.root.title("Лабораторна робота №3")

        tools = ["Крапка", "Лінія", "Прямокутник", "Еліпс"]
        for idx, tool in enumerate(tools):
            prefix = "✓ " if tool == self.current_tool else "   "
            self.obj_menu.entryconfig(idx, label=f"{prefix}{tool}")

        for name, btn in self.tool_buttons.items():
            if name == self.current_tool:
                btn.config(relief=tk.SUNKEN, bg="#d0d0d0")
            else:
                btn.config(relief=tk.RAISED, bg="#f0f0f0")

    def setup_menu(self):
        menubar = tk.Menu(self.root)
        self.root.config(menu=menubar)

        file_menu = tk.Menu(menubar, tearoff=0)
        menubar.add_cascade(label="Файл", menu=file_menu)

        self.obj_menu = tk.Menu(menubar, tearoff=0)
        tools = ["Крапка", "Лінія", "Прямокутник", "Еліпс"]
        for tool in tools:
            self.obj_menu.add_command(
                label=tool,
                command=lambda t=tool: self.set_tool(t)
            )
        menubar.add_cascade(label="Об'єкти", menu=self.obj_menu)

        help_menu = tk.Menu(menubar, tearoff=0)
        menubar.add_cascade(label="Довідка", menu=help_menu)

    def setup_toolbar(self):
        toolbar = tk.Frame(self.root, bd=1, relief=tk.RAISED, bg="#f0f0f0")

        tools = [
            ("Крапка", "Малювання крапки"),
            ("Лінія", "Малювання лінії"),
            ("Прямокутник", "Малювання прямокутника"),
            ("Еліпс", "Малювання еліпса")
        ]
```

```
        for name, hint in tools:
            btn = tk.Button(
                toolbar, text=name, command=lambda n=name: self.set_tool(n),
                width=12, relief=tk.RAISED
            )
            btn.pack(side=tk.LEFT, padx=2, pady=2)
            ToolTip(btn, hint)
            self.tool_buttons[name] = btn

        toolbar.pack(side=tk.TOP, fill=tk.X)

    def redraw_all(self):
        self.canvas.delete("all")
        for shape in self.shapes:
            shape.draw(self.canvas)

    def add_shape(self, shape):
        if len(self.shapes) >= self.max_shapes:
            self.shapes.pop(0)
        self.shapes.append(shape)
        self.redraw_all()

    def on_press(self, event):
        self.start_x = event.x
        self.start_y = event.y

        if self.current_tool == "Крапка":
            self.add_shape(PointShape(event.x, event.y, event.x, event.y))

    def on_drag(self, event):
        if self.current_tool == "Крапка":
            self.add_shape(PointShape(event.x, event.y, event.x, event.y))
            return

        if self.start_x is None:
            return

        self.redraw_all()

        if self.current_tool == "Лінія":
            self.rubber_id = self.canvas.create_line(
                self.start_x, self.start_y, event.x, event.y, fill="black")
        elif self.current_tool == "Прямокутник":
            self.rubber_id = self.canvas.create_rectangle(
                self.start_x, self.start_y, event.x, event.y, outline="black")
```

```
        elif self.current_tool == "Еліпс":
            dx = abs(event.x - self.start_x)
            dy = abs(event.y - self.start_y)
            self.rubber_id = self.canvas.create_oval(
                self.start_x - dx, self.start_y - dy,
                self.start_x + dx, self.start_y + dy,
                outline="black")

    def on_release(self, event):
        if self.current_tool == "Крапка":
            self.start_x = None
            self.start_y = None
            return

        if self.start_x is None:
            return

        shape = None
        if self.current_tool == "Лінія":
            shape = LineShape(self.start_x, self.start_y, event.x, event.y)
        elif self.current_tool == "Прямокутник":
            shape = RectShape(self.start_x, self.start_y, event.x, event.y)
        elif self.current_tool == "Еліпс":
            shape = EllipseShape(self.start_x, self.start_y, event.x, event.y)

        if shape:
            self.add_shape(shape)

        self.start_x = None
        self.start_y = None

if __name__ == "__main__":
    main_window = tk.Tk()
    main_window.geometry("800x600")
    app = EditorApp(main_window)
    main_window.mainloop()
```

shapes.py

```
class Shape:
    def __init__(self, x1, y1, x2, y2):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2
```

```
    def draw(self, canvas):
        pass

class PointShape(Shape):
    def draw(self, canvas):
        r = 4
        canvas.create_oval(
            self.x2 - r, self.y2 - r,
            self.x2 + r, self.y2 + r,
            fill="black", outline="black"
        )

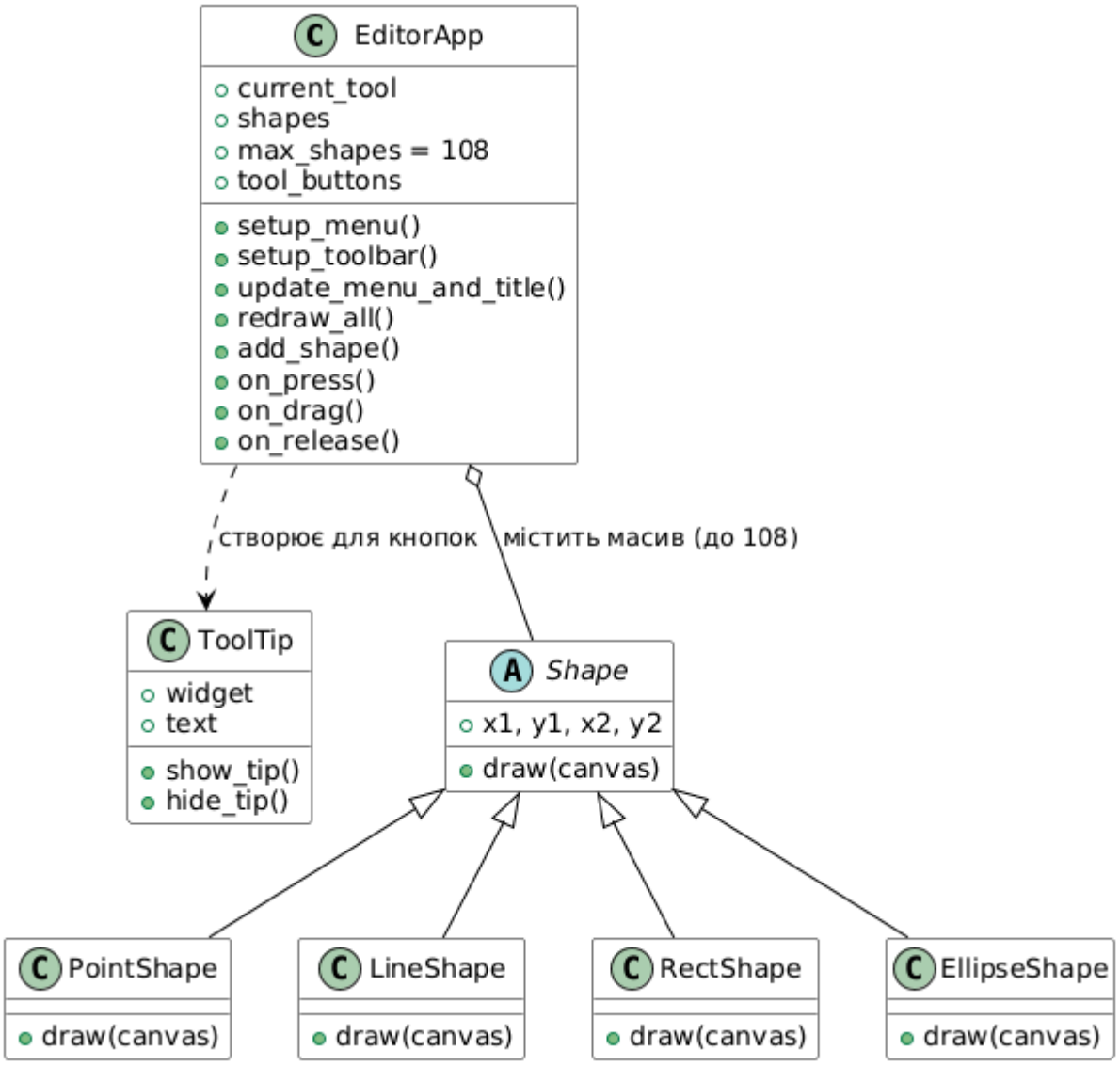
class LineShape(Shape):
    def draw(self, canvas):
        canvas.create_line(self.x1, self.y1, self.x2, self.y2, fill="black")

class RectShape(Shape):
    def draw(self, canvas):
        canvas.create_rectangle(
            self.x1, self.y1, self.x2, self.y2,
            outline="black", fill=""
        )

class EllipseShape(Shape):
    def draw(self, canvas):
        dx = abs(self.x2 - self.x1)
        dy = abs(self.y2 - self.y1)
        canvas.create_oval(
            self.x1 - dx, self.y1 - dy,
            self.x1 + dx, self.y1 + dy,
            outline="black", fill="lightgreen"
        )
```

Виконання роботи

Схема



Висновок

У цій лабораторній роботі я виконала завдання згідно зі своїм варіантом. Мною було розроблено програму, яка є простим графічним редактором, реалізованим в об’єктно-орієнтованому стилі. Під час виконання роботи я практично закріпила та навчилася використовувати такі принципи ООП, як інкапсуляція, абстракція типів, успадкування та поліморфізм